MPI and related concepts

Flynn's taxonomy for concurrent processing

Single instruction stream, (SISD)
single data stream
- classic sequential code

Multiple inst streams, (MIMD)
multiple data streams
- web browser / server

Mult inst streams, (MISD)
single data stream
- no clue - multiple analyses?

Single inst stream, (SIMD)
multiple data streams
- MPI

Some details on MPI

- designed to work with C
- nice Java layer that we're using in Kotlin
- most documentation is in C, but that's ok, it's all close

```
fun main(args: Array<String>) {

    MPI.Init(args)

    val size = MPI.COMM_WORLD.getSize()
    val rank = MPI.COMM_WORLD.getRank()

    if (size < 2) {
        println("Too few processes!")
        System.exit(1)
    }

    val buffer = MPI.newIntBuffer(1)

    val numItemsToTransfer = 1
    val sourceRank = 0
    val destinationRank = 1
    val messageTag = 0  // supplemental tag, often just
    var bufferPosition = 0

    if (rank == 0) {
        val valueToTransmit = 5
        buffer.put(bufferPosition, valueToTransmit)
        MPI.COMM_WORLD.send(buffer, numItemsToTransfer,
                            destinationRank, messageTag
    } else if (rank == 1) {
        val status = MPI.COMM_WORLD.recv(buffer, numIte
                                         MPI.INT, sourc
        val valueReceived = buffer.get(bufferPosition)
        println("Process 1 received value $valueReceive
    }

    MPI.Finalize()
}
```

# of proces

unique id
this one

every proce
its own b

break symmetry

ses

for
e

ss has
uffer

not used

```
 MPI.INT,
)

msToTransfer,
eRank, messageTag)

d from process 0")
```

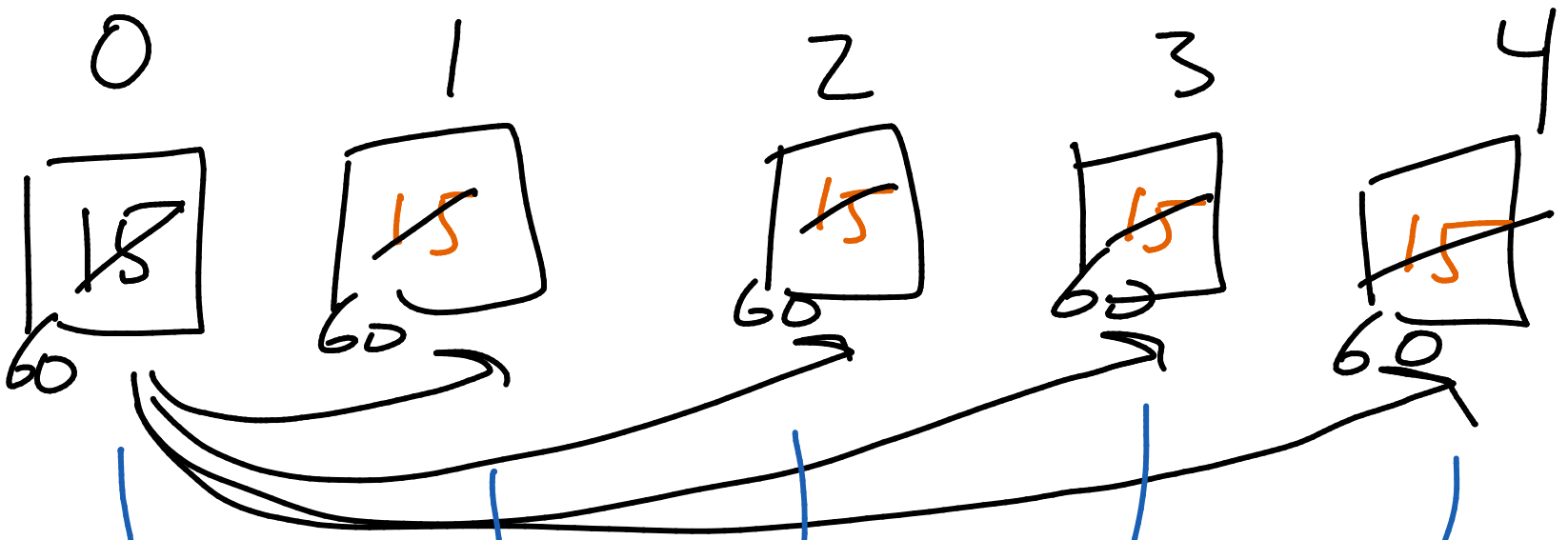
send/recv are both blocking

recv blocks / waits until something has been sent

send blocks until someone is ready to receive

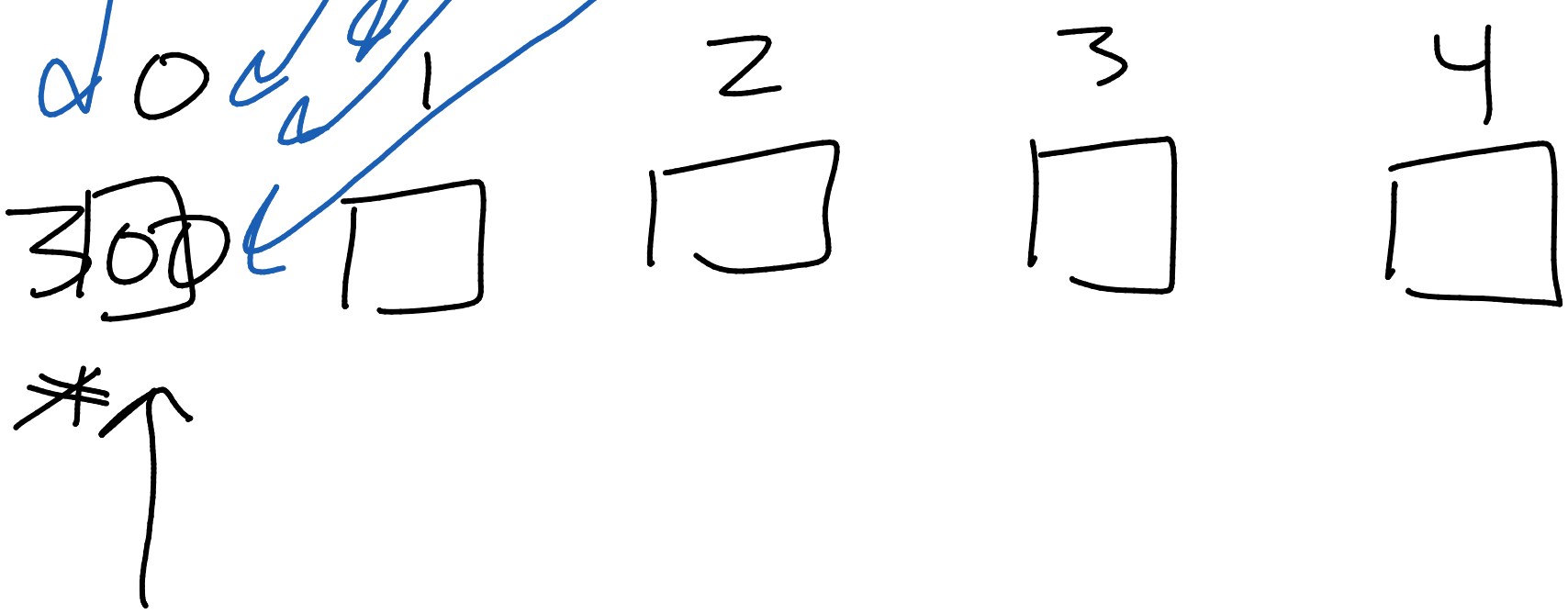
```
import java.nio.*;

fun main(args: Array<String>) {

    MPI.Init(args)

    val size = MPI.COMM_WORLD.getSize()
    val rank = MPI.COMM_WORLD.getRank()
    val name = MPI.COMM_WORLD.getName()

    val buffer = MPI.newIntBuffer(1)

    val numItemsToTransfer = 1
    val sourceRank = 0
    val messageTag = 0  // supplemental tag, often just not used
    var bufferPosition = 0
    val numberToPlayWith = 5

    if (rank == sourceRank) {
        buffer.put(bufferPosition, numberToPlayWith * 3)
    }

    MPI.COMM_WORLD.bcast(buffer, numItemsToTransfer, MPI.INT, sourceRan

    val valueReceived = buffer.get(0)

    val valueModified = valueReceived * 4

    buffer.put(bufferPosition, valueModified)

    val finalAnswer = MPI.newIntBuffer(1)
    val destProcess = 0
    MPI.COMM_WORLD.reduce(buffer, finalAnswer, numItemsToTransfer,
                          MPI.INT, MPI.SUM, destProcess)

    if (rank == 0) {
        println("Final answer = ${finalAnswer.get(0)}")
    }

    MPI.Finalize()
}
```

Five processes (e.g.) buffer



barrier
(all processes wait to proceed until they all get here)

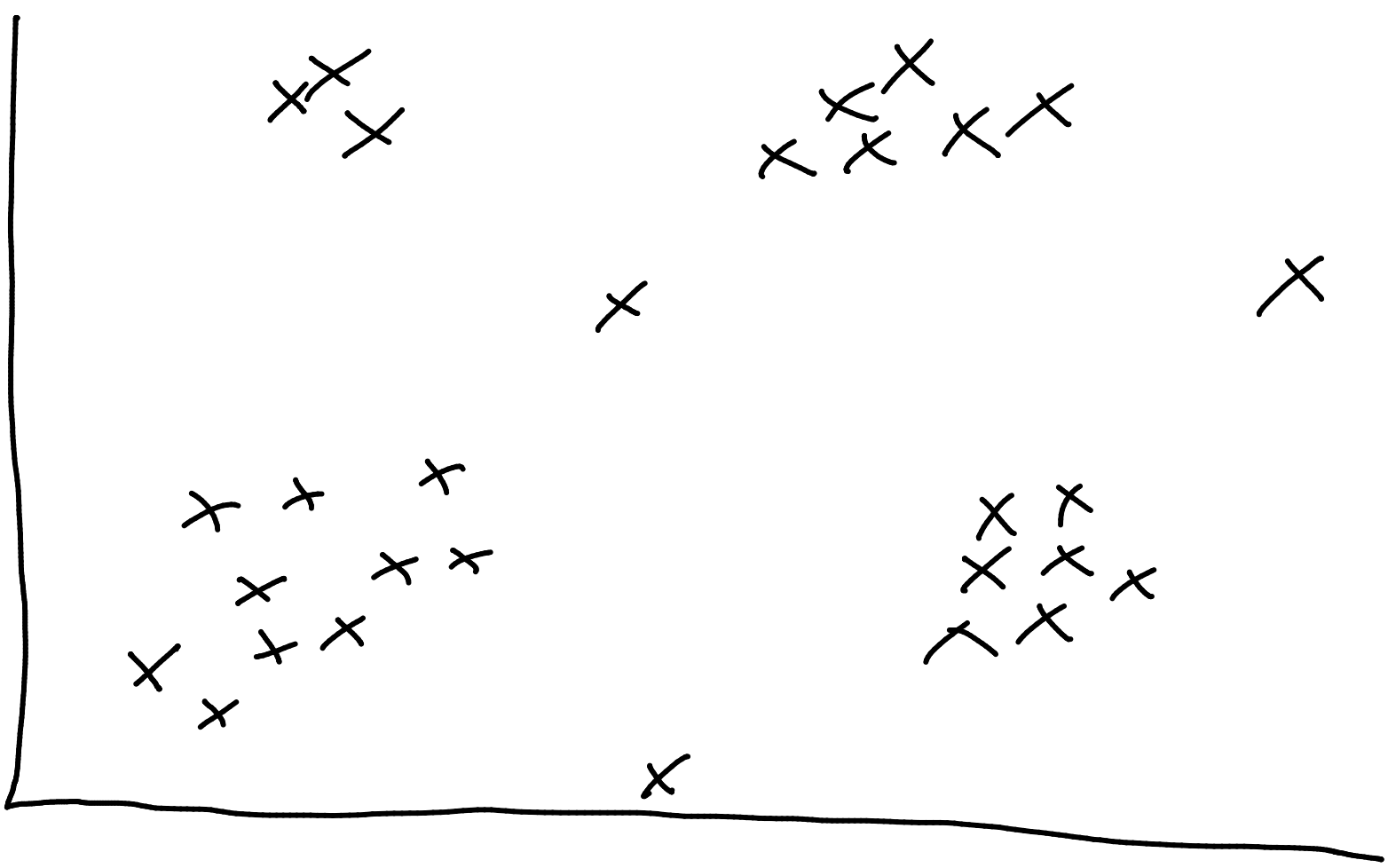
finalAnswer



K-means clustering

- algorithm you'll implement in MPI
- I'm going to give you serial code to do this

Purpose: find clusters in data



A clustering algorithm finds cluster centers, which are prototype values for each cluster

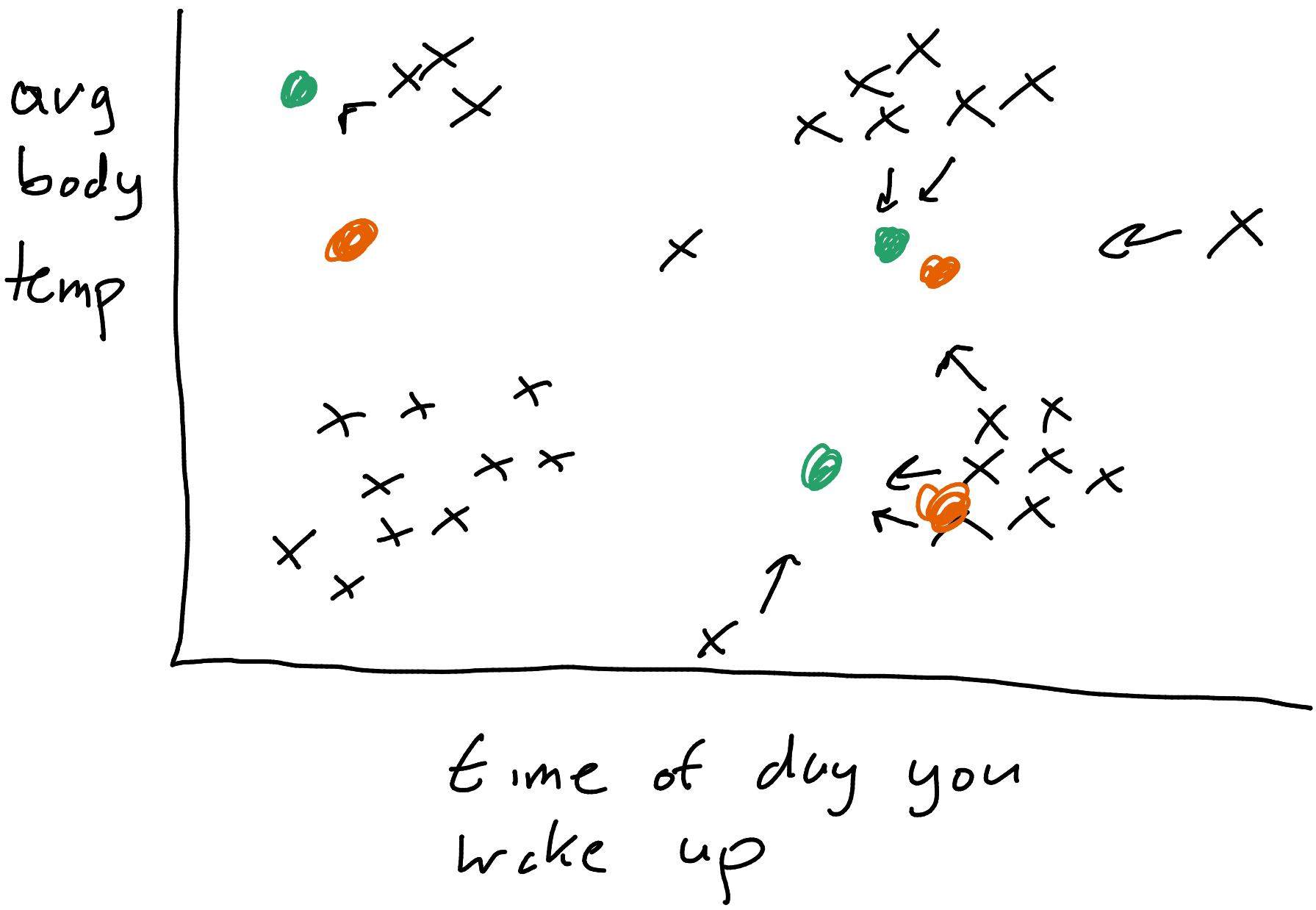
K-means - start off w/ a number of clusters (K) you're looking for

E.g. pick K = 3

Process

Initialize a set of starting centers.

- pick 3 cluster centers to start (at random)
- green = start centers



Repeat:

1. assign all points to closest center
2. create new cntrs, each of which is the average of all points assigned to same center

⋮ until it doesn't change much

- green = round 1
- round 2 (orange)

Output = cluster cntrs

cluster error = sum of distances of all points to their nearest centers

cluster error is guaranteed not to increase*

so stop when cluster error changes by a small amount

---

how do we measure distance?

usually, we use Euclidean distance (but not here)

which is

(x_1, y_1) (x_2, y_2)

$$\text{Euc dist} = \sqrt{(x_2 - x_1)^2 + (y_1 - y_2)^2}$$

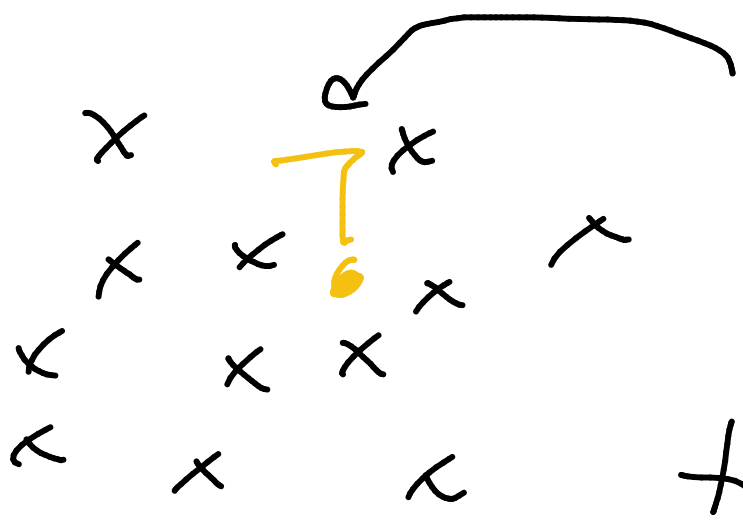
Instead, we use <u>squared</u> <u>Euclidean</u> distance

$$= (x_2 - x_1)^2 + (y_2 - y_1)^2$$

Why?

Step 2 we average all the points in a cluster to get the new center.

Mathematically, the average minimizes total squared Euc distance of all points to the center



where is the center that minimizes total distance of points to that center?

avg - if you're using squred Euc distance